

Resumo Completo: Capítulo 5 - Entrada/Saída

Introdução: O Papel do Sistema Operacional na E/S

O Sistema Operacional (SO) é responsável por gerenciar todos os dispositivos de Entrada/Saída (E/S) de um computador. Suas principais funções nesse contexto são:

- **Emitir comandos** para os dispositivos.
 - **Interceptar interrupções** geradas por eles.
 - **Tratar erros** de hardware e software.
 - **Fornecer uma interface de abstração** simples, uniforme e fácil de usar entre os dispositivos e o restante do sistema, independentemente do hardware específico.
-

5.1 Princípios do Hardware de E/S

Esta seção aborda a E/S da perspectiva do hardware, focando em como ele é programado.

5.1.1 Dispositivos de E/S: Classificação Geral

Os dispositivos de E/S podem ser divididos em duas categorias principais:

- **Dispositivos de Blocos (Block Devices):**
 - **Definição:** Armazenam informações em blocos de tamanho fixo (ex: 512 bytes a 65.536 bytes), cada um com seu próprio endereço.
 - **Propriedade Essencial:** Cada bloco pode ser lido ou escrito de forma independente dos outros (acesso aleatório).
 - **Exemplos:** Discos rígidos (HDs), Discos de Estado Sólido (SSDs), discos Blu-ray e pendrives.
- **Dispositivos de Caractere (Character Devices):**
 - **Definição:** Enviam ou aceitam um fluxo de caracteres (stream), sem qualquer estrutura de bloco.
 - **Propriedade Essencial:** Não são endereçáveis e não possuem a operação de "busca" (seek).
 - **Exemplos:** Impressoras, interfaces de rede, mouses, teclados.

Observação: Essa classificação não é perfeita. Dispositivos como relógios (que apenas geram interrupções) e telas mapeadas em memória não se encaixam bem em nenhuma das categorias.

5.1.2 Controladores de Dispositivos (Adaptadores)

- **Estrutura:** As unidades de E/S consistem em um componente **mecânico** (o dispositivo em si) e um componente **eletrônico** (o **controlador** ou **adaptador**).
- **Função do Controlador:**
 1. Recebe comandos abstratos do SO (ex: "ler bloco X").
 2. Converte esses comandos em sinais elétricos de baixo nível para o dispositivo.
 3. Converte o fluxo serial de bits vindo do dispositivo em um bloco de bytes para a CPU.
 4. Realiza a verificação e correção de erros (usando **ECC - Error-Correcting Code**).

- Armazena temporariamente os dados em um **buffer interno** antes de transferi-los para a memória principal.

5.1.3 Comunicação entre CPU e Controladores

Existem duas maneiras principais para a CPU se comunicar com os registradores de controle e buffers de dados dos dispositivos:

- **1. Portas de E/S (Espaço de E/S Separado):**
 - **Como funciona:** Cada registrador de controle recebe um "número de porta" único. A CPU usa instruções especiais de E/S (como **IN** e **OUT** na arquitetura x86) para ler e escrever nessas portas.
 - **Característica:** Os espaços de endereçamento de memória e de E/S são completamente separados. O endereço de memória **4** é diferente da porta de E/S **4**.
 - **Desvantagem:** Requer o uso de linguagem de montagem (assembly) para acessar as portas, pois linguagens de alto nível como C não possuem essas instruções.
- **2. E/S Mapeada na Memória (Memory-Mapped I/O):**
 - **Como funciona:** Todos os registradores de controle e buffers dos dispositivos são mapeados em endereços do espaço de memória principal.
 - **Característica:** Não existem instruções de E/S especiais. Os registradores são lidos e escritos como se fossem variáveis comuns na memória.
 - **Vantagens:**
 - Os drivers de dispositivo podem ser escritos inteiramente em C.
 - Qualquer instrução que acessa a memória pode ser usada nos registradores.
 - O gerenciamento de proteção é simplificado, pois o SO pode simplesmente não mapear as páginas de memória dos dispositivos no espaço de um processo de usuário.
 - **Desvantagens:**
 - **Problema de Cache:** Se um registrador de dispositivo for armazenado em cache, o SO lerá o valor antigo do cache em vez do valor atual do hardware. A solução é o hardware permitir a desabilitação seletiva do cache para certas páginas de memória.
 - **Complexidade em Múltiplos Barramentos:** Em sistemas com um barramento de memória de alta velocidade separado, os dispositivos de E/S podem não "ver" os endereços. Isso exige hardware adicional (como "snooping") para encaminhar os acessos.
- **Esquema Híbrido:** A arquitetura x86 usa um modelo híbrido, com portas de E/S para registradores de controle e E/S mapeada na memória para o buffer de dados da placa de vídeo (RAM de vídeo).

5.1.4 Acesso Direto à Memória (DMA - Direct Memory Access)

- **Problema:** A E/S programada e orientada a interrupções (byte a byte) consome muito tempo de CPU.
- **Solução:** Usar um **controlador de DMA**, um chip especial que gerencia a transferência de blocos de dados entre um dispositivo e a memória principal **sem a intervenção da CPU**.
- **Processo de Operação do DMA (leitura de disco):**

1. A CPU programa o controlador de DMA com:
 - Endereço de memória para onde os dados irão.
 - Número de bytes a serem transferidos.
 - Porta do dispositivo de E/S.
 - Direção da transferência (leitura ou escrita).
2. A CPU comanda o controlador de disco para ler os dados do disco para seu buffer interno.
3. Quando os dados estão no buffer do disco, o controlador de DMA inicia a transferência.
4. O DMA transfere os dados diretamente do buffer do disco para a memória principal.
5. Após cada palavra transferida, o DMA incrementa o endereço de memória e decrementa o contador de bytes.
6. Quando o contador chega a zero, o DMA gera **uma única interrupção** para a CPU, sinalizando que a transferência do bloco inteiro foi concluída.

- **Modos de Operação do DMA:**

- **Roubo de Ciclo (Cycle Stealing):** O DMA transfere uma palavra de cada vez, "roubando" ciclos do barramento da CPU.
- **Modo de Surto (Burst Mode):** O DMA adquire o controle do barramento e transfere um bloco inteiro de uma vez. É mais eficiente para o DMA, mas pode bloquear a CPU por mais tempo.

5.1.5 Interrupções Revisitadas

- **Interrupção Precisa:** Deixa a máquina em um estado bem definido e consistente.
 1. O PC (Program Counter) é salvo em um local conhecido.
 2. Todas as instruções *antes* daquela apontada pelo PC foram concluídas.
 3. Nenhuma instrução *depois* daquela apontada pelo PC foi concluída (ou seus efeitos foram desfeitos).
 4. O estado da instrução apontada pelo PC é conhecido.
 - **Interrupção Imprecisa:** Ocorre em máquinas com arquiteturas complexas (pipeline, superescalar) e deixa a máquina em um estado inconsistente, com múltiplas instruções em diferentes estágios de execução. Isso torna o trabalho do SO muito mais difícil para reiniciar o processo interrompido.
-

5.2 Princípios do Software de E/S

5.2.1 Objetivos do Software de E/S

1. **Independência de Dispositivo:** Ser capaz de escrever programas que acessem qualquer dispositivo de E/S sem precisar especificar o dispositivo de antemão. Ex: um programa que lê um arquivo deve funcionar se a entrada vier de um HD, DVD ou pendrive.
2. **Nomeação Uniforme:** O nome de um arquivo ou dispositivo deve ser uma simples cadeia de caracteres, independente do dispositivo. No UNIX, por exemplo, todos os dispositivos podem ser integrados no sistema de arquivos.
3. **Tratamento de Erros:** Os erros devem ser tratados o mais próximo possível do hardware. Se o controlador não consegue resolver, o driver tenta. Apenas se as camadas mais baixas falharem, o erro é reportado para as camadas superiores.
4. **Transferências Síncronas (Bloqueantes) vs. Assíncronas (Não-bloqueantes):**

- **Síncrona:** O processo que inicia a E/S é bloqueado até que a operação seja concluída. É mais fácil de programar.
- **Assíncrona:** O processo continua sua execução enquanto a E/S ocorre em segundo plano. O SO notifica o processo quando a E/S termina (via interrupção).
- O SO geralmente faz com que a E/S, que é fisicamente assíncrona, pareça síncrona para os processos de usuário.

5. **Utilização de Buffers (Buffering):** Dados são armazenados temporariamente em um buffer na memória enquanto são transferidos entre dispositivos. Isso é crucial para lidar com diferenças de velocidade e para evitar perda de dados.

6. **Dispositivos Compartilhados vs. Dedicados:** O SO deve ser capaz de gerenciar o acesso a ambos os tipos. Discos são compartilhados, enquanto impressoras são dedicadas a um único usuário por vez. O gerenciamento de dispositivos dedicados pode levar a impasses (deadlocks).

5.2.2 Formas de Realizar a E/S

1. E/S Programada:

- A CPU faz todo o trabalho. Ela copia os dados para o dispositivo e entra em um laço de **espera ocupada (busy waiting)** ou **polling**, verificando continuamente o status do dispositivo até que ele esteja pronto para o próximo dado.
- **Vantagem:** Simples de implementar.
- **Desvantagem:** Extremamente ineficiente, desperdiça 100% do tempo de CPU.

2. E/S Orientada a Interrupções:

- A CPU inicia a transferência de dados e fica livre para executar outro processo.
- Quando o dispositivo termina sua tarefa, ele gera uma **interrupção**.
- A CPU para o que está fazendo, salva seu contexto e executa uma **rotina de tratamento de interrupção (Interrupt Service Routine - ISR)** que processa os dados e possivelmente inicia a próxima transferência.
- **Vantagem:** Muito mais eficiente que a E/S programada, permite a multiprogramação.
- **Desvantagem:** Para transferências de grandes volumes de dados, a sobrecarga de uma interrupção por byte/caractere ainda é alta.

3. E/S usando DMA:

- É a forma mais eficiente. A CPU programa o controlador de DMA para transferir um bloco inteiro de dados e só é interrompida **uma vez**, no final da transferência completa.
- **Vantagem:** Libera a CPU quase que totalmente do trabalho de E/S, resultando na menor sobrecarga.

5.3 Camadas do Software de E/S

O software de E/S é geralmente organizado em quatro camadas:

1. Tratadores de Interrupção (Camada mais baixa):

- **Função:** Esconder os detalhes da interrupção do resto do SO. A rotina de interrupção é executada, e o driver que estava bloqueado esperando pela E/S é desbloqueado e colocado na

fila de processos prontos.

2. Drivers de Dispositivo:

- **Função:** Contêm todo o código específico para um determinado dispositivo ou classe de dispositivos. São geralmente escritos pelo fabricante do hardware.
- **Operação:** Recebem solicitações abstratas (ex: "ler bloco N") do software independente de dispositivo, traduzem para comandos concretos para o controlador, programam o hardware e gerenciam a fila de solicitações para aquele dispositivo.

3. Software de E/S Independente do Dispositivo (Camada do SO):

- **Função:** Realizar as funções de E/S que são comuns a todos os dispositivos e fornecer uma interface uniforme para o software de usuário.
- **Tarefas Típicas:**
 - **Interface uniforme para drivers:** Define um padrão para como os drivers devem se parecer.
 - **Nomeação de dispositivos:** Mapeia nomes simbólicos (ex: `/dev/disk0`) para o driver correto.
 - **Proteção de dispositivos:** Verifica as permissões de acesso.
 - **Utilização de buffers:** Gerencia o buffering de dados.
 - **Alocação de dispositivos:** Gerencia o uso de dispositivos dedicados.
 - **Relatório de erros:** Propaga erros das camadas inferiores.

4. Software de E/S no Nível do Usuário (Camada mais alta):

- **Função:** Consiste em bibliotecas e programas que realizam E/S.
- **Exemplos:**
 - **Funções de biblioteca:** `printf` e `scanf` em C são rotinas de biblioteca que formatam dados e então fazem chamadas de sistema (como `write`) para realizar a E/S.
 - **Spooling:** Um processo especial (daemon) gerencia uma fila de requisições para um dispositivo dedicado (como uma impressora). Um processo de usuário não escreve diretamente na impressora, mas sim em um arquivo em um **diretório de spooling**. O daemon então imprime os arquivos da fila um por um. Isso evita que um processo "prenda" o dispositivo.

5.4 Discos

5.4.1 Hardware do Disco

- **Discos Magnéticos:** Organizados em **cilindros**, **trilhas** e **setores**. O tempo de acesso é a soma do **tempo de busca** (mover o braço), **atraso de rotação** (esperar o setor chegar sob a cabeça) e **tempo de transferência**.
- **Geometria de Disco:**
 - **Geometria Física:** A organização real dos setores. Discos modernos usam **gravação por zona (zone bit recording)**, com mais setores nas trilhas externas.
 - **Geometria Virtual:** Uma geometria falsa (cilindro, cabeça, setor) que o disco apresenta ao SO para manter a compatibilidade. O controlador do disco faz o mapeamento interno.

- **LBA (Logical Block Addressing):** Um método mais moderno onde os setores são simplesmente numerados de 0 em diante, escondendo completamente a geometria.
- **RAID (Redundant Array of Independent Disks):**
 - **Ideia Central:** Combinar múltiplos discos para parecerem um único disco lógico, melhorando desempenho, confiabilidade ou ambos.
 - **Níveis de RAID:**
 - **RAID 0 (Striping):** Distribui os dados entre os discos. Ótimo desempenho, mas **nenhuma redundância**. Uma falha de disco perde todos os dados.
 - **RAID 1 (Mirroring):** Duplica os dados em discos espelhados. Excelente confiabilidade e bom desempenho de leitura, mas custo de 50% do espaço.
 - **RAID 3:** "Rasga" os dados em nível de byte e usa um disco dedicado para paridade. Requer discos sincronizados.
 - **RAID 4:** "Rasga" os dados em nível de bloco (faixas) e usa um disco de paridade dedicado. O disco de paridade pode se tornar um gargalo.
 - **RAID 5:** Como o RAID 4, mas distribui a paridade entre todos os discos. Elimina o gargalo do disco de paridade e é uma das configurações mais populares. Tolerância a falha de um disco.
 - **RAID 6:** Como o RAID 5, mas com dois blocos de paridade independentes. Tolerância a falha de até dois discos simultaneamente, oferecendo maior confiabilidade.

5.4.2 Formatação e Gerenciamento de Disco

- **Formatação de Baixo Nível:** Cria as trilhas e setores no disco, escrevendo preâmbulos e ECCs.
- **Particionamento:** Divide o disco em uma ou mais partições lógicas. O **MBR (Master Boot Record)** no setor 0 contém a tabela de partições e o código de inicialização. O **GPT (GUID Partition Table)** é um padrão mais novo que supera as limitações do MBR.
- **Formatação de Alto Nível:** Cria um sistema de arquivos em uma partição (ex: lista de blocos livres, diretório raiz).

5.4.3 Algoritmos de Escalonamento de Braço de Disco

O objetivo é minimizar o tempo total de busca atendendo a uma fila de solicitações.

- **FCFS (First-Come, First-Served):** Atende as solicitações na ordem em que chegam. Justo, mas muito ineficiente.
- **SSF (Shortest Seek First):** Atende a solicitação mais próxima da posição atual do braço. Otimiza o movimento do braço, mas pode levar à **inanição (starvation)** de solicitações nos extremos do disco.
- **Algoritmo do Elevador (SCAN):** O braço se move em uma direção (ex: para dentro), atendendo todas as solicitações em seu caminho. Ao chegar ao extremo, ele inverte a direção. É mais justo que o SSF.
- **C-SCAN (Circular SCAN):** Similar ao Elevador, mas após chegar ao extremo, o braço retorna ao início sem atender solicitações no caminho de volta, e então começa a varrer novamente na mesma direção. Oferece tempos de espera mais uniformes.

5.4.4 Tratamento de Erros e Armazenamento Estável

- **Setores Defeituosos (Bad Sectors):**

- O controlador pode lidar com eles de forma transparente, substituindo um setor defeituoso por um **reserva (spare)**.
 - O SO pode gerenciar uma lista de blocos defeituosos, marcando-os como "em uso" em um arquivo especial para que não sejam alocados.
 - **Armazenamento Estável:**
 - Um ideal onde uma operação de escrita ou é concluída com sucesso ou não faz nada, mesmo com falhas de energia.
 - Uma implementação simples usa dois discos idênticos. Uma **escrita estável** consiste em:
 1. Escrever o bloco no disco 1.
 2. Ler de volta para verificar.
 3. Se correto, escrever o bloco no disco 2.
 4. Ler de volta para verificar.
 - Após uma falha, um programa de recuperação compara os dois discos e conserta inconsistências.
-

5.5 Relógios e Temporizadores

- **Hardware de Relógio:** Geralmente consiste em um **oscilador de cristal** de alta precisão, um **contador** que é decrementado em cada pulso, e um **registrador de apoio** para carregar o contador. Quando o contador chega a zero, ele gera uma interrupção (**tique do relógio**).
 - **Software de Relógio (Driver do Relógio):**
 1. **Manter a Hora do Dia:** Incrementa um contador em memória a cada tique.
 2. **Preempção de Processos:** Decrementa o quantum do processo em execução. Quando chega a zero, chama o escalonador.
 3. **Contabilizar o Uso da CPU:** Mantém estatísticas sobre o tempo de CPU usado por cada processo.
 4. **Gerenciar Alarmes:** O SO mantém uma lista de alarmes solicitados por processos e gera um sinal quando o tempo expira.
-

5.6 Interfaces com o Usuário

5.6.1 Software de Entrada

- **Teclado:**
 - O hardware gera uma interrupção para cada tecla pressionada e liberada, enviando um **código de varredura (scan code)**, não um caractere ASCII.
 - O driver do teclado interpreta os códigos de varredura para determinar o caractere, considerando teclas como SHIFT e CTRL.
 - **Modo Cru (Raw Mode):** O driver passa cada caractere para o programa imediatamente.
 - **Modo Canônico (Cooked Mode):** O driver processa a entrada, acumulando caracteres em um buffer de linha e lidando com edição (backspace, apagar linha) antes de passar a linha completa para o programa.
- **Mouse:**
 - Relata movimentos relativos (Δx , Δy) e o estado dos botões. O software é responsável por traduzir isso em movimentos de cursor na tela e eventos de clique (simples e duplo).

5.6.2 Software de Saída

- **Janelas de Texto:**
 - Drivers de terminais suportam **sequências de escape** (ex: padrão ANSI) para controlar o cursor, limpar a tela, mudar cores, etc. Isso permite a criação de editores e interfaces de texto mais complexas.
 - **Interfaces Gráficas do Usuário (GUI):**
 - **Paradigma WIMP:** Windows (Janelas), Icons (Ícones), Menus e Pointing device (Dispositivo Apontador).
 - **Sistema X Window (UNIX/Linux):**
 - Arquitetura cliente-servidor. O **servidor X** executa na máquina do usuário (controla a tela/teclado), e o **cliente X** (a aplicação) pode executar local ou remotamente.
 - O protocolo X define a comunicação entre eles.
 - Bibliotecas como **Xlib**, toolkits como **Intrinsics** e ambientes como **Gnome (GTK+)** e **KDE (Qt)** são construídos sobre o X.
 - **Windows GDI (Graphics Device Interface):**
 - Um conjunto de rotinas para desenhar texto e gráficos vetoriais.
 - **Bitmaps:** Representações de imagens como uma grade de pixels. A operação **BitBlt** é usada para mover blocos de pixels rapidamente.
 - **Fontes:** Fontes modernas como **TrueType** são baseadas em contornos (vetores), permitindo escalonamento para qualquer tamanho sem perda de qualidade, ao contrário das fontes de bitmap.
 - **Telas de Toque:**
 - **Telas Resistivas:** Detectam pressão. Suportam qualquer tipo de stylus, mas geralmente não suportam múltiplos toques.
 - **Telas Capacitivas:** Detectam a interrupção de um campo elétrico (pelo dedo). Suportam **toques múltiplos (multitouch)**, mas requerem um objeto condutivo.
-

5.7 Clientes Magros (Thin Clients)

- **Conceito:** Um computador simples, com software mínimo (geralmente um navegador ou cliente de terminal), que depende de um servidor central para a maior parte do processamento e armazenamento.
 - **Vantagens:**
 - **Manutenção Centralizada:** Mais fácil de atualizar e gerenciar.
 - **Backups Centralizados:** Mais confiáveis.
 - **Menor Custo:** O hardware do cliente pode ser mais barato.
 - **Exemplo:** Chromebook, que executa ChromeOS e depende de aplicativos web.
-

5.8 Gerenciamento de Energia

- **Objetivo:** Reduzir o consumo de energia para diminuir custos (em desktops) e aumentar a vida útil da bateria (em portáteis).
- **Estratégias do SO:**
 - **Monitor:** Desligar a retroiluminação (backlight) após um período de inatividade. Em telas avançadas, pode-se iluminar apenas zonas específicas da tela.

- **Disco Rígido:** Parar a rotação do disco (**spin down**) após inatividade. A decisão de quando parar é um trade-off: parar economiza energia, mas reiniciar o disco consome tempo e um pico de energia.
- **CPU:**
 - Colocar a CPU em estados de baixo consumo ("dormir") quando estiver ociosa.
 - **Escalonamento Dinâmico de Voltagem e Frequência (DVFS):** Reduzir a velocidade do relógio e a voltagem da CPU. Executar mais devagar pode ser mais eficiente energeticamente, pois o consumo de energia é proporcional ao quadrado da voltagem.
- **Memória:** Colocar o sistema em modo de **hibernação**, salvando o conteúdo da RAM no disco e desligando a energia da RAM completamente.
- **Comunicação Sem Fio:** Desligar o rádio e acordá-lo periodicamente para verificar se há mensagens recebidas, que são armazenadas em buffer na estação-base.